Carrasco Quiñones Diego Alexander

Bitácora práctica 09

Parte 0 · Ver el problema

Antes de modificar cualquier código reproduje el problema para dejar registrado el comportamiento inicial del sistema. Para ello seguí los cuatro puntos de esta parte.

Punto 1 · Apagar el servicio notificacion

Para comenzar apagué el servicio de notificaciones con el comando `docker compose stop notificacion`. En la Figura 1 se observa primero el listado de contenedores obtenido con `docker compose ps`, en el que todos aparecen activos y saludables, y después el comando de apagado con el que el contenedor notificacion quedó detenido.

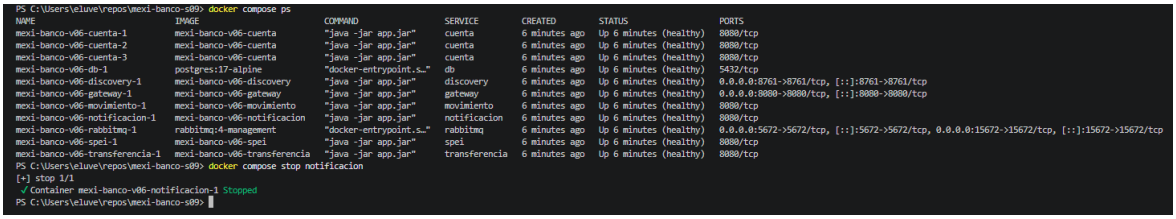


Figura 1. *Listado de contenedores con docker compose ps y apagado del contenedor notificacion.*

Punto 2 · Hacer una transferencia por el gateway

Después realicé una transferencia a través del gateway con la ayuda de dos comandos de PowerShell. El primero crea un diccionario con la CLABE de la cuenta que envía y la CLABE de la cuenta que recibe junto con el monto de 50 y lo convierte a texto JSON para guardarlo en la variable `$body`.

El segundo comando utiliza `Invoke-RestMethod` para hacer una petición HTTP a la API. Con `-Uri` se indica la dirección del endpoint de transferencias en el puerto 8080 que corresponde al gateway y con `-Method Post` se señala que se envía información para crear un recurso. Además `-ContentType "application/json"` avisa al servidor que el cuerpo viene en formato JSON mientras que `-Body $body` adjunta el JSON generado por el primer comando.

La Figura 2 muestra ambos comandos junto con el resultado: un error HTTP 500 del servidor remoto. La transferencia respondió con ese error porque el servicio de notificaciones estaba detenido y aunque el cargo y el abono ya se habían realizado quedó en estado ABONADA sin completar el proceso de notificación. Con esta solicitud se pidió transferir 50 de la cuenta de origen a la de destino por medio del gateway que escucha en el puerto 8080.

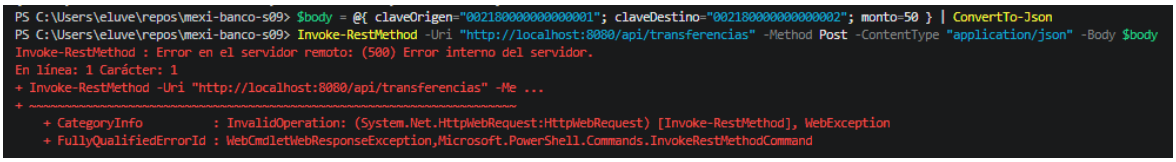


Figura 2. *Creación del cuerpo JSON y llamada a /api/transferencias que termina con el error HTTP 500.*

Punto 3 · Consultar el estado y los saldos

Consulté el estado de la transferencia y los saldos de las cuentas para comprobar que el dinero se movió aunque la respuesta fue un error. La transferencia quedó en estado ABONADA y respondió con un error HTTP 500 porque el servicio de notificaciones estaba detenido y aunque el dinero ya se había transferido el proceso no pudo completar el envío de la notificación.

En la Figura 3 se ve la consulta con `curl.exe` al endpoint de transferencias filtrando por el estado ABONADA. La respuesta devuelve la transferencia con ID 1 en estado ABONADA y con el campo `intentosDeCompensacion` en 0. La Figura 4 es un acercamiento a esa misma respuesta donde se distinguen con claridad el estado y el saldo de origen tras el cargo.



Figura 3. *Consulta con curl.exe de las transferencias en estado ABONADA con la respuesta HTTP 200 y la transferencia con id 1.*

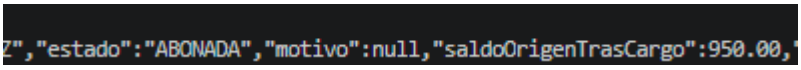


Figura 4. *Acercamiento a la respuesta anterior donde se distinguen el estado ABONADA y el saldoOrigenTrasCargo de 950.00.*

La Figura 5 presenta el estado de los saldos de Ana y de Beto y la siguiente tabla los compara con los saldos iniciales.



Figura 5. *Consulta de las cuentas de Ana y de Beto con curl.exe y sus saldos de 950.00 y 550.00.*

Cuenta	Saldo inicial	Saldo actual	Cambio
Ana (origen)	$1,000.00	$950.00	−$50.00
Beto (destino)	$500.00	$550.00	+$50.00

Punto 4 · Volver a encender notificacion

Una vez comprobado el problema volví a encender el servicio con `docker compose start notificacion`. En la Figura 6 se observa que Docker confirma primero que los servicios discovery y rabbitmq y la base de datos db están saludables y que después inicia el contenedor notificacion.

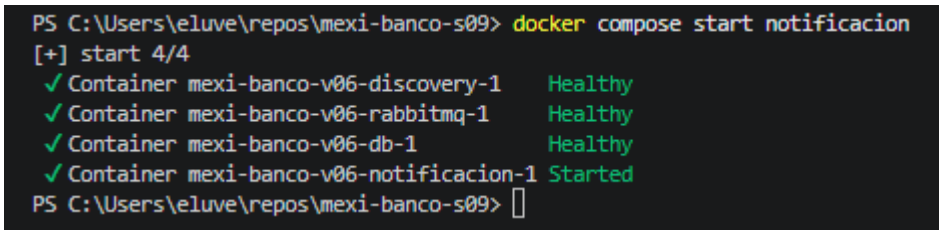


Figura 6. *Encendido del servicio con docker compose start notificacion.*

Parte 1 · Publicar el evento

Punto 1 · Declarar el exchange transferencias como @Bean

En este paso creé la clase `RabbitConfig.java` para configurar el exchange transferencias de RabbitMQ mediante Spring. Utilicé la anotación `@Configuration` que indica que la clase contiene configuraciones de la aplicación y la anotación `@Bean` que registra el exchange para que Spring lo administre.

Declaré un exchange de tipo topic llamado transferencias que es durable y que no se elimina automáticamente cuando deja de utilizarse. Este exchange recibe los eventos generados por el servicio de transferencias y los dirige a las colas correspondientes según su clave de enrutamiento (routing key).

Esta configuración es importante porque permite que el servicio de transferencias publique eventos en RabbitMQ sin comunicarse directamente por HTTP con el servicio de notificaciones y de esta manera ambos servicios pueden trabajar de forma más independiente. En este punto únicamente declaré el exchange ya que la publicación de eventos y la configuración de las colas se realizan en los siguientes pasos.

La Figura 7 muestra el archivo con el bean del exchange y la siguiente tabla explica la función de cada elemento del código.

```
PS C:\Users\eluve\repos\mexi-banco-s09> Get-Content .\transferencia\src\main\java\mx\mexibanco\transferencia\mensajeria\RabbitConfig.java
package mx.mexibanco.transferencia.mensajeria;

import org.springframework.amqp.core.TopicExchange;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitConfig {

    @Bean
    public TopicExchange transferenciasExchange() {
        return new TopicExchange("transferencias", true, false);
    }
}
PS C:\Users\eluve\repos\mexi-banco-s09>
```

Figura 7. *Clase RabbitConfig.java con el bean del exchange transferencias de tipo topic.*

Elemento	Función
`@Configuration`	Indica que esta clase contiene configuraciones que Spring debe procesar.
`public class RabbitConfig`	Define la clase donde se configura RabbitMQ.
`@Bean`	Indica que el objeto devuelto por el método se registra y se administra como un componente de Spring.
`TopicExchange`	Representa un exchange de tipo topic que permite dirigir mensajes según patrones de claves de enrutamiento.
`"transferencias"`	Establece el nombre del exchange.
`true`	Indica que el exchange es durable y por lo tanto su definición permanece después de reiniciar RabbitMQ.
`false`	Indica que el exchange no debe eliminarse automáticamente cuando deja de utilizarse.

Punto 2 · Configurar un RabbitTemplate con convertidor JSON

En este paso modifiqué la clase `RabbitConfig.java` para configurar el componente `RabbitTemplate` y el convertidor `Jackson2JsonMessageConverter` sin perder la configuración del exchange transferencias creada en el punto anterior.

Agregué un bean de tipo `Jackson2JsonMessageConverter` que se encarga de convertir los objetos Java en mensajes con formato JSON. Después configuré el `RabbitTemplate` con `ConnectionFactory` para establecer la conexión con RabbitMQ y con `setMessageConverter()` para asignarle el convertidor JSON.

Esta configuración permite que el servicio de transferencias publique eventos en RabbitMQ de una manera más sencilla porque los objetos que representan los eventos se convierten automáticamente a JSON y ese formato facilita que otros servicios reciban e interpreten la información del mensaje. Con este cambio quedó listo el mecanismo para publicar los eventos

aunque todavía no se enviaba ningún mensaje porque eso ocurre al implementar la publicación en los siguientes pasos.

En la Figura 8 aparece el archivo con el exchange y el bean del convertidor JSON y en la Figura 9 el bean del `RabbitTemplate` que utiliza ese convertidor.

```
PS C:\Users\eluve\repos\mexi-banco-s09> Get-Content .\transferencia\src\main\java\mx\mexibanco\transferencia\mensajeria\RabbitConfig.java
package mx.mexibanco.transferencia.mensajeria;

import org.springframework.amqp.core.TopicExchange;
import org.springframework.amqp.rabbit.connection.ConnectionFactory;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.amqp.support.converter.Jackson2JsonMessageConverter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class RabbitConfig {

    @Bean
    public TopicExchange transferenciasExchange() {
        return new TopicExchange("transferencias", true, false);
    }

    @Bean
    public Jackson2JsonMessageConverter jackson2JsonMessageConverter() {
        return new Jackson2JsonMessageConverter();
    }
```

Figura 8. *RabbitConfig.java con el exchange y el bean del convertidor Jackson2JsonMessageConverter.*

```
    @Bean
    public RabbitTemplate rabbitTemplate(
            ConnectionFactory connectionFactory,
            Jackson2JsonMessageConverter conversorJson) {

        RabbitTemplate rabbitTemplate = new RabbitTemplate(connectionFactory);
        rabbitTemplate.setMessageConverter(conversorJson);

        return rabbitTemplate;
    }
}
PS C:\Users\eluve\repos\mexi-banco-s09>
```

Figura 9. *Bean del RabbitTemplate configurado con el convertidor JSON.*

Punto 3 · Crear el record del evento

En este paso creé el archivo `TransferenciaEvento.java` dentro del paquete `mx.mexibanco.transferencia.mensajeria` y definí en él un record de Java que establece la estructura de los mensajes enviados a RabbitMQ.

El contrato contiene nueve campos que se muestran en la Figura 10 y se resumen en la tabla. Para representar los importes utilicé `BigDecimal` y para registrar la fecha y la hora utilicé `Instant` de acuerdo con los tipos de datos que ya emplea el proyecto.

```
PS C:\Users\eluve\repos\mexi-banco-s09> Get-Content .\transferencia\src\main\java\mx\mexibanco\transferencia\mensajeria\TransferenciaEvento.java
package mx.mexibanco.transferencia.mensajeria;

import java.math.BigDecimal;
import java.time.Instant;

public record TransferenciaEvento(
        String eventoId,
        String tipo,
        Long transferenciaId,
        String clabeOrigen,
        String clabeDestino,
        BigDecimal monto,
        BigDecimal saldoOrigen,
        BigDecimal saldoDestino,
        Instant fecha
) {
}
PS C:\Users\eluve\repos\mexi-banco-s09>
```

Figura 10. *Record TransferenciaEvento con los nueve campos del contrato.*

Campo	Tipo
`eventoId`	`String`
`tipo`	`String`
`transferenciaId`	`Long`
`clabeOrigen`	`String`
`clabeDestino`	`String`
`monto`	`BigDecimal`
`saldoOrigen`	`BigDecimal`
`saldoDestino`	`BigDecimal`
`fecha`	`Instant`

El campo `eventoId` identifica cada evento y permite mantener el mismo identificador cuando es necesario volver a publicarlo mientras que el campo `tipo` distingue entre los eventos de una transferencia realizada y los de una transferencia revertida.

Este contrato establece una estructura común para la información que viaja entre los servicios mediante RabbitMQ y de esta forma el servicio de transferencias puede generar mensajes con los datos necesarios para que otros servicios los interpreten y los procesen. En este punto únicamente definí la estructura del evento y falta integrarla con la lógica de la saga para publicar los mensajes cuando una transferencia alcance los estados correspondientes.

Punto 4 · Publicar TransferenciaRealizada y TransferenciaRevertida

Cuando la saga llega a COMPLETADA el servicio guarda el estado y publica el evento `TransferenciaRealizada` en el exchange transferencias con la clave `transferencia.realizada`. Cuando el abono falla y la devolución sale bien la transferencia queda en COMPENSADA y se publica `TransferenciaRevertida` con la clave `transferencia.revertida`.

Los dos casos usan el mismo método `publicarEvento` y por esa razón el reintento programado de compensaciones también publica el evento. El servicio guarda primero el estado y publica después. Además las pruebas unitarias verifican ambas publicaciones con el `RabbitTemplate`.

La Figura 11 muestra una búsqueda en `TransferenciaService` con los números de línea donde se llama a `publicarEvento` con cada clave y donde el método envía el evento al exchange con `convertAndSend`. La Figura 12 corresponde al registro de la transferencia 12 donde se ven los estados por los que pasó y la publicación de los dos eventos con sus claves. En ese registro aparece también una advertencia porque no se pudo consultar el saldo final de destino.

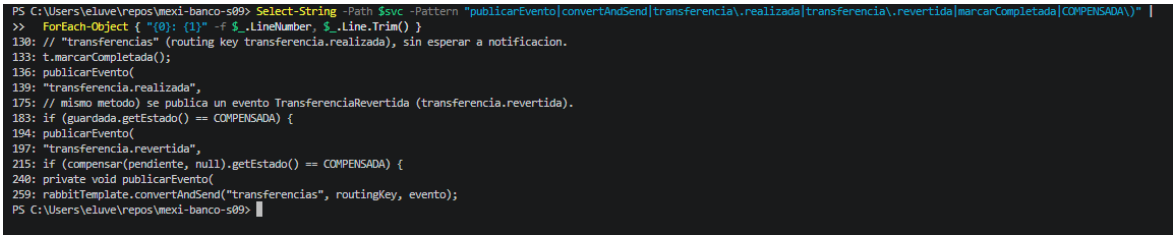


Figura 11. *Búsqueda en TransferenciaService de los puntos donde se publican los eventos con sus claves y de la llamada convertAndSend al exchange transferencias.*

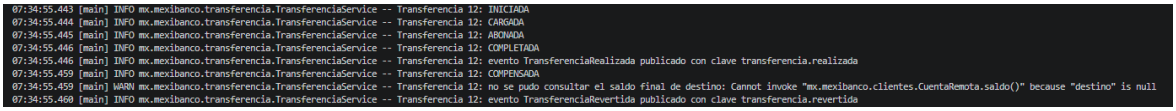


Figura 12. *Registro de la transferencia 12 con sus estados y la publicación de los eventos TransferenciaRealizada y TransferenciaRevertida.*

Punto 5 · Quitar la llamada HTTP a notificacion

En este punto eliminé de la transferencia la llamada HTTP al servicio de notificaciones. `TransferenciaService` ya no depende de `NotificacionCliente` y tampoco hace el POST síncrono a `/notificaciones`. Antes cuando notificacion estaba caída la excepción salía después del abono y la fila se quedaba en ABONADA mientras que el cliente recibía un error aunque su dinero ya había llegado. Ahora la transferencia se marca como COMPLETADA sin esperar a notificacion y solo publica un evento. El servicio Spei se mantuvo como estaba con su propia copia del cliente HTTP.

La Figura 13 muestra la búsqueda de las referencias a `NotificacionCliente` y la comparación de cambios donde se retiran la dependencia y las llamadas a notificaciones y se agrega el `RabbitTemplate` con el método `publicarEvento`.

```
PS C:\Users\eluve\repos\mexi-banco-s09> Get-ChildItem -Recurse .\transferencia\src\main\java\mx\mexibanco\transferencia -Include *.java |
>>   Select-String -Pattern "NotificacionCliente"
PS C:\Users\eluve\repos\mexi-banco-s09> Select-String -Path $svc -Pattern "notificacion" -CaseSensitive:$false |
>>   ForEach-Object { "{0}: {1}" -f $_.LineNumber, $_.Line.Trim() }
120: // el error. Terminarlo sin que el cliente espere a notificacion es justo lo que resuelve pasar
126: // LAB S09: aqui transferencia espera a notificacion por HTTP. Si notificacion esta caida, la
130: // "transferencias" (routing key transferencia.realizada), sin esperar a notificacion.
PS C:\Users\eluve\repos\mexi-banco-s09> git diff -- $svc | Select-String "^[-+].*(notificaciones|Notificacion|publicarEvento|rabbitTemplate)"

+import org.springframework.amqp.rabbit.core.RabbitTemplate;
-import mx.mexibanco.clientes.NotificacionCliente;
-       private final NotificacionCliente notificaciones;
+       private final RabbitTemplate rabbitTemplate;
-                                                  MovimientoCliente movimientos, NotificacionCliente notificaciones,
+                          RabbitTemplate rabbitTemplate) {
+              this.rabbitTemplate = rabbitTemplate;
-              this.notificaciones = notificaciones;
-              notificaciones.notificar(cargo);
-              notificaciones.notificar(abono);
+              publicarEvento(
+                     publicarEvento(
+       private void publicarEvento(
+    rabbitTemplate.convertAndSend("transferencias", routingKey, evento);


PS C:\Users\eluve\repos\mexi-banco-s09>
```

Figura 13. *Búsqueda de NotificacionCliente y comparación de cambios (git diff) en TransferenciaService con el retiro de la llamada HTTP y la incorporación del RabbitTemplate.*

Para comprobarlo dejé notificacion apagado y realicé una transferencia de 10. En la Figura 14 se ve que respondió HTTP 201 y quedó en COMPLETADA (id 8) y la Figura 15 repite el final de esa respuesta junto con la comprobación de que el contenedor notificacion seguía detenido. Como todavía no existe una cola el mensaje publicado se pierde y por eso esto se resuelve en la Parte 2.



Figura 14. *Transferencia de 10 con notificacion apagado y su respuesta HTTP 201 con estado COMPLETADA.*



Figura 15. *Final de la misma respuesta HTTP 201 y estado detenido del contenedor notificacion con docker compose ps -a.*

También activé los *publisher confirms* con la propiedad `spring.rabbitmq.publisher-confirm-type` que en la Figura 16 aparece con el valor `correlated` en el `application.yml`. La consulta a la API de RabbitMQ muestra además el canal de la conexión con `confirm` en True.

```
PS C:\Users\eluve\repos\mexi-banco-s09> Select-String -Path .\transferencia\src\main\resources\application.yml -Pattern "publisher-confirm-type"

transferencia\src\main\resources\application.yml:32:    publisher-confirm-type: correlated


PS C:\Users\eluve\repos\mexi-banco-s09> (Invoke-RestMethod -Headers $h -Uri "http://localhost:15672/api/channels") | Select-Object name, confirm, connection_details | Format-List


name               : 172.22.0.6:44422 -> 172.22.0.3:5672 (1)
confirm            : True
connection_details : @{name=172.22.0.6:44422 -> 172.22.0.3:5672; peer_host=172.22.0.6; peer_port=44422}



PS C:\Users\eluve\repos\mexi-banco-s09>
```

Figura 16. *Propiedad publisher-confirm-type con valor correlated en application.yml y canal de RabbitMQ con confirm en True.*

¿Qué garantiza la confirmación? Garantiza que el broker recibió el mensaje y se hizo responsable de él. Dicho de otra forma el productor no pierde el mensaje por un fallo de red entre la aplicación y RabbitMQ.

¿Qué no garantiza? La confirmación no cubre tres situaciones:

- **Que el mensaje llegó a una cola.** Con el exchange sin bindings el broker confirma igualmente y descarta el mensaje y eso se ve en esta práctica con `publish_in: 2` y ninguna cola.
- **Que alguien lo consumió.** Confirmar no es lo mismo que entregar ni que procesar y de eso se encarga el ack del consumidor que se implementa en la Parte 2.
- **Que el aviso se registró en el código.** Con `correlated` el broker responde pero el código todavía no tiene un `ConfirmCallback` y por eso un nack pasaría sin que nadie lo note.

Parte 2 · Consumir el evento

En el servicio notificacion ya existía un esqueleto en `mensajeria/ConsumidorDeTransferencias.java` marcado con `// LAB S09`.

Punto 1 · Declarar la cola notificacion.transferencias y su binding

En el servicio notificacion creé la clase `RabbitConfig` en la que declaré tres elementos como `@Bean`. El primero es el exchange transferencias de tipo topic y durable con los mismos parámetros que en el servicio de transferencia. El segundo es la cola durable `notificacion.transferencias` y el tercero es el binding que las une mediante el patrón `transferencia.*`. Con ese patrón la cola recibe los dos eventos que publica transferencia que son `transferencia.realizada` y `transferencia.revertida`. También agregué una copia del record `TransferenciaEvento` porque notificacion es un servicio aparte y no comparte clases con transferencia.

La Figura 17 muestra las líneas de la clase donde se declaran el exchange la cola y el binding.

```
PS C:\Users\eluve\repos\mexi-banco-s09> Select-String -Path .\notificacion\src\main\java\mx\mexibanco\notificacion\mensajeria\RabbitConfig.java -Pattern "EXCHANGE|COLA|PATRON|@Bean" | ForEach-Object { "{0}: {1}" -f $_.LineNumber, $_.Line.Trim() }
7: import org.springframework.amqp.core.TopicExchange;
16: public static final String EXCHANGE = "transferencias";
17: public static final String COLA = "notificacion.transferencias";
18: public static final String PATRON = "transferencia.*";
20: // Mismo exchange que declara transferencia, con los mismos parametros (topic, durable).
21: @Bean
22: public TopicExchange transferenciasExchange() {
23: return new TopicExchange(EXCHANGE, true, false);
26: @Bean
27: public Queue colaNotificaciones() {
28: return QueueBuilder.durable(COLA).build();
31: @Bean
32: public Binding bindingNotificaciones(Queue colaNotificaciones, TopicExchange transferenciasExchange) {
33: return BindingBuilder.bind(colaNotificaciones).to(transferenciasExchange).with(PATRON);
37: @Bean
PS C:\Users\eluve\repos\mexi-banco-s09>
```

Figura 17. *RabbitConfig del servicio notificacion con las constantes del exchange, la cola y el patrón y sus tres métodos @Bean.*

Para comprobarlo compilé el módulo con el resultado BUILD SUCCESS y regeneré el jar para después reconstruir el contenedor y consultar RabbitMQ. En el panel de administración la cola `notificacion.transferencias` aparece en estado running y sin mensajes (Figura 18) y en su detalle se ve que no tiene consumidores y que está enlazada al exchange transferencias con la clave `transferencia.*` (Figura 19).

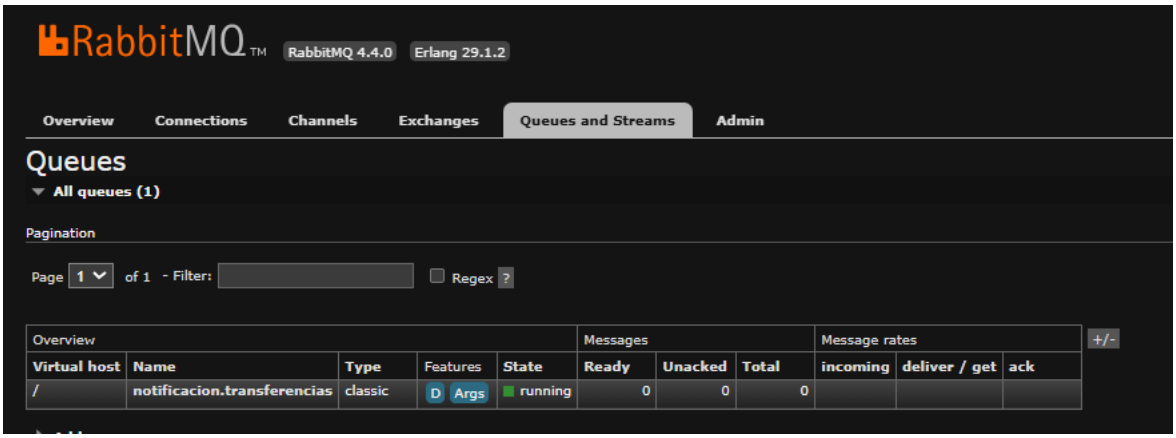


Figura 18. *Panel de RabbitMQ con la cola notificacion.transferencias de tipo classic en estado running y sin mensajes.*

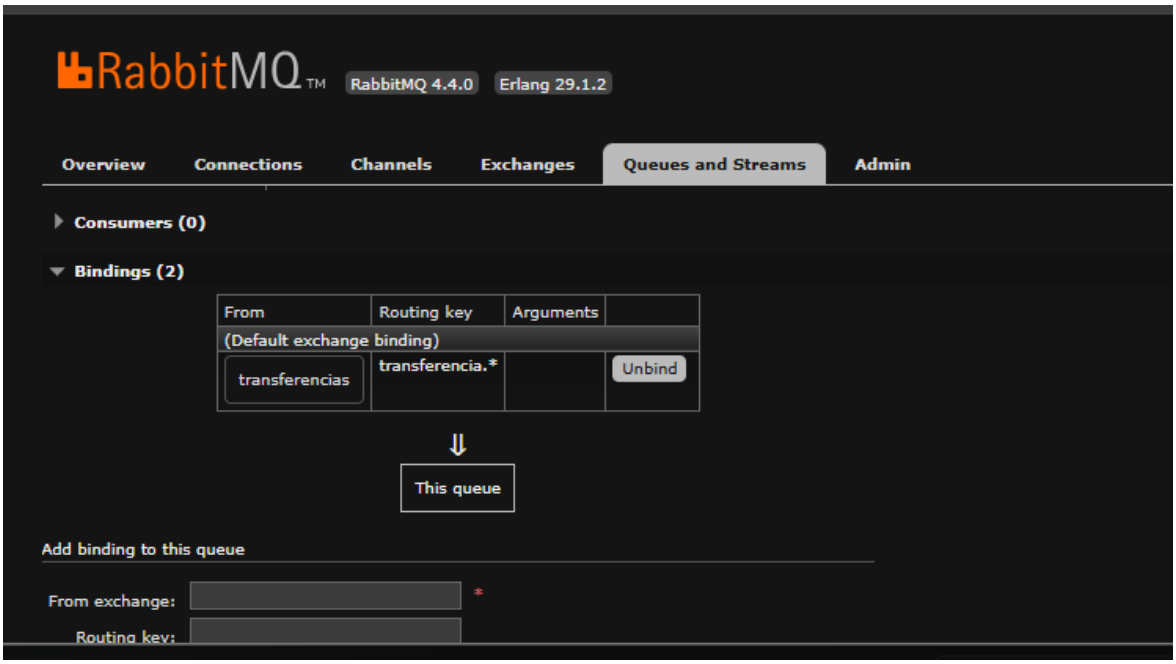


Figura 19. *Detalle de la cola en el panel con Consumers (0) y el binding del exchange transferencias con la clave transferencia.*.*

La consulta a la API de RabbitMQ de la Figura 20 confirma que la cola es durable y que tiene 0 mensajes y 0 consumidores aunque en esa captura la consulta de bindings todavía no devuelve filas. En la Figura 21 la consulta de bindings del exchange ya muestra el enlace hacia la cola con el patrón `transferencia.*`.

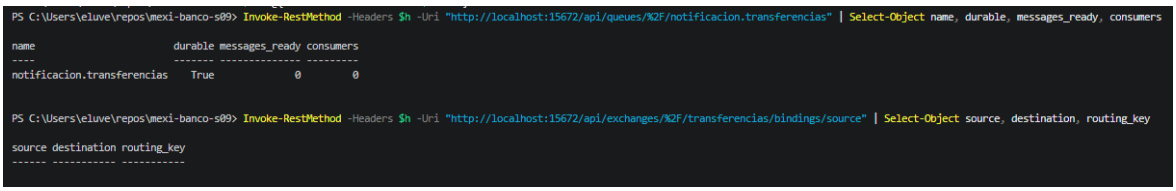


Figura 20. *Consulta a la API de RabbitMQ con la cola durable sin mensajes ni consumidores y consulta de bindings que aún no devuelve filas.*

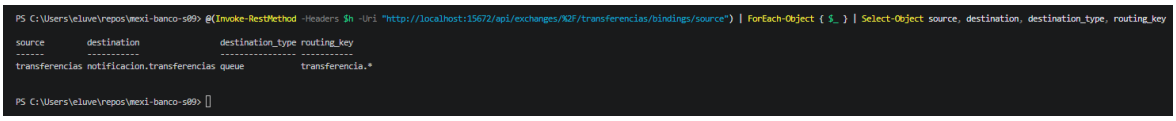


Figura 21. *Consulta a la API de RabbitMQ con el binding del exchange transferencias hacia la cola notificacion.transferencias con la clave transferencia.*.*

Punto 2 · Escribir el @RabbitListener que guarda los avisos

En `ConsumidorDeTransferencias` escribí un `@RabbitListener` sobre la cola `notificacion.transferencias`. Por cada evento llama a `NotificacionService.avisarTransferencia` que guarda los avisos en la bandeja. Se guardan dos avisos por cada `TransferenciaRealizada` (uno para la cuenta de origen y otro para la de destino) y uno por cada `TransferenciaRevertida` (para la cuenta de origen con el texto "Tu transferencia no se realizó y te devolvimos el dinero"). Los avisos de un mismo evento se guardan en una sola transacción.

La Figura 22 muestra las líneas del consumidor donde aparecen el listener sobre la cola y el orden entre guardar y confirmar. Además activé `acknowledge-mode: manual` en `application.yml` como se ve en la Figura 23 y esa configuración se explica en el punto 3.

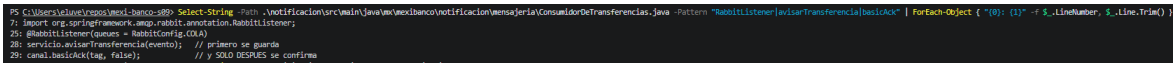


Figura 22. *Líneas del consumidor con el @RabbitListener sobre la cola, el llamado a avisarTransferencia y el basicAck posterior.*

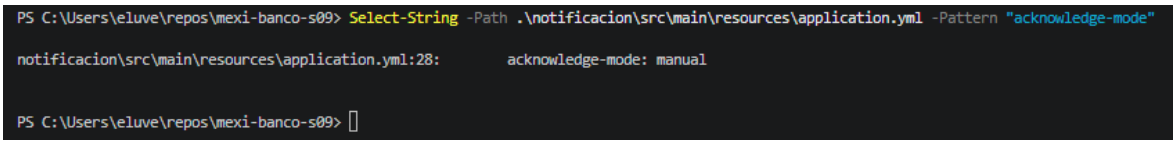


Figura 23. *Propiedad acknowledge-mode: manual en el application.yml de notificacion.*

Avisos de TransferenciaRealizada (dos por evento)

Para comprobar estos avisos apagué notificacion y realicé tres transferencias con los ids 9 a 11. Las tres respondieron 201 y quedaron en COMPLETADA y el panel de RabbitMQ mostró 3 mensajes Ready en `notificacion.transferencias` sin consumidores (Figura 24 y Figura 25). Al encender notificacion la cola bajó a 0 con un consumidor activo. En el registro aparece cada evento guardado y después confirmado (Figura 26) y la bandeja de cada cuenta tiene los tres avisos: Enviaste en la de origen y Recibiste en la de destino (Figura 27).

Esto corresponde al desacople en el tiempo porque el productor y el consumidor ya no tienen que estar vivos al mismo tiempo.

```
PS C:\Users\eluve\repos\mexi-banco-s09> $body = @{ claveOrigen = "00218000000000001"; claveDestino = "00218000000000002"; monto = 10.00 } | ConvertTo-Json
PS C:\Users\eluve\repos\mexi-banco-s09> 1..3 | ForEach-Object {
>>   try {
>>     $r = Invoke-WebRequest -UseBasicParsing -Method Post -Uri "http://localhost:8080/api/transferencias" -ContentType "application/json" -Body $body
>>     $j = $r.Content | ConvertFrom-Json
>>     "Intento ${_}: HTTP $($r.StatusCode)  id=$($j.id)  estado=$($j.estado)"
>>   } catch {
>>     "Intento ${_}: HTTP " + [int]$_.Exception.Response.StatusCode
>>   }
>> }
Intento 1: HTTP 201  id=9  estado=COMPLETADA
Intento 2: HTTP 201  id=10  estado=COMPLETADA
Intento 3: HTTP 201  id=11  estado=COMPLETADA
PS C:\Users\eluve\repos\mexi-banco-s09> Invoke-RestMethod -Headers $h -Uri "http://localhost:15672/api/queues/%2F/notificacion.transferencias" | Select-Object name, messages_ready, consumers

name                       messages_ready consumers
----                       -------------- ---------
notificacion.transferencias              3         0


PS C:\Users\eluve\repos\mexi-banco-s09>
```

Figura 24. *Tres transferencias con notificacion apagado (ids 9 a 11) y consulta de la cola con 3 mensajes listos y 0 consumidores.*

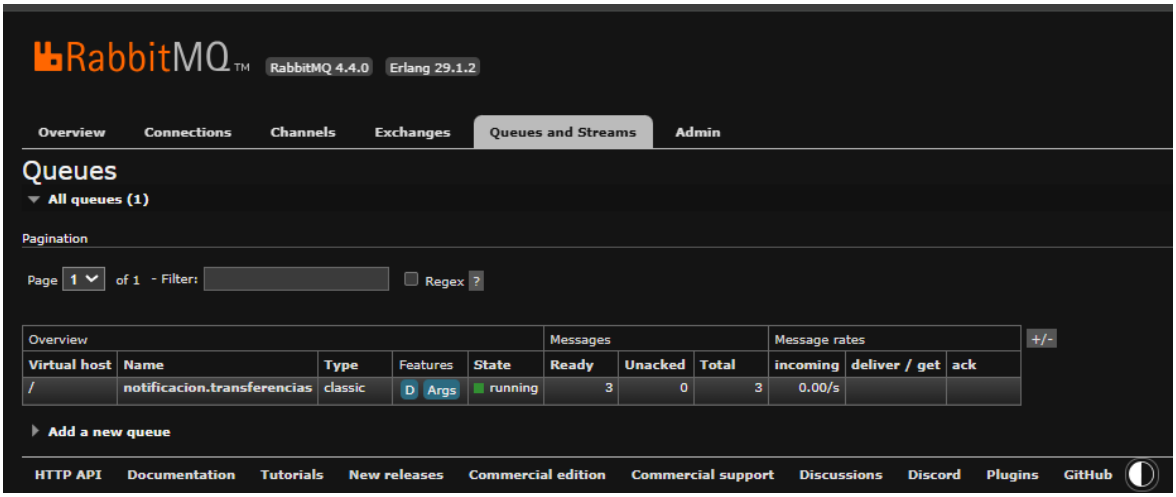


Figura 25. *Panel de RabbitMQ con 3 mensajes Ready en la cola notificacion.transferencias.*

```
PS C:\Users\eluve\repos\mexi-banco-s09> docker compose logs --since 2m notificacion | Select-String "procesado y confirmado|Enviaste|Recibiste|ERROR"

notificacion-1  | 2026-11-08T15:59:10.106Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService    : [notificacion] 00218000000000001 - Enviaste 10 a la cuenta 00218000000000002, saldo resultante 650.00
notificacion-1  | 2026-11-08T15:59:10.210Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService    : [notificacion] 00218000000000002 - Recibiste 10 de la cuenta 00218000000000001, saldo resultante 710.00
notificacion-1  | 2026-11-08T15:59:10.224Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias      : [notificacion] evento transferencia-9-TransferenciaRealizada procesado y confirmado
notificacion-1  | 2026-11-08T15:59:10.225Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService    : [notificacion] 00218000000000001 - Enviaste 10 a la cuenta 00218000000000002, saldo resultante 640.00
notificacion-1  | 2026-11-08T15:59:10.227Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService    : [notificacion] 00218000000000002 - Recibiste 10 de la cuenta 00218000000000001, saldo resultante 720.00
notificacion-1  | 2026-11-08T15:59:10.231Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias      : [notificacion] evento transferencia-10-TransferenciaRealizada procesado y confirmado
notificacion-1  | 2026-11-08T15:59:10.232Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService    : [notificacion] 00218000000000001 - Enviaste 10 a la cuenta 00218000000000002, saldo resultante 630.00
notificacion-1  | 2026-11-08T15:59:10.235Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.notificacion.NotificacionService    : [notificacion] 00218000000000002 - Recibiste 10 de la cuenta 00218000000000001, saldo resultante 730.00
notificacion-1  | 2026-11-08T15:59:10.239Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias      : [notificacion] evento transferencia-11-TransferenciaRealizada procesado y confirmado
```

Figura 26. *Registro de notificacion con los avisos Enviaste y Recibiste y la confirmación de los eventos de las transferencias 9, 10 y 11.*

```
PS C:\Users\eluve\repos\mexi-banco-s09> $a = Invoke-RestMethod "http://localhost:8080/api/notificaciones?clabe=00218000000000001"
PS C:\Users\eluve\repos\mexi-banco-s09> $a | Select-Object -First 5 | Format-Table id, clabeCuenta, mensaje, fecha -AutoSize -Wrap

id clabeCuenta        mensaje                                                
```

Aviso de TransferenciaRevertida (uno por evento)

Activé la falla provocada de la S08 con `FALLAR_ABONO_A_CLABE=002180000000000002` y realicé una transferencia hacia esa cuenta. El abono falló a propósito y la saga devolvió el dinero a la cuenta de origen por lo que la transferencia 12 quedó en COMPENSADA y el servicio respondió 502 con el detalle del motivo. Al llegar a COMPENSADA transferencia publicó `TransferenciaRevertida` con la clave `transferencia.revertida` y notificacion consumió el evento y guardó el aviso "Tu transferencia no se realizó y te devolvimos el dinero" en la bandeja de la cuenta de origen. Solo después hizo el ack. Así el aviso de reversión llega aunque el cliente nunca vio el abono algo que antes no existía.

La Figura 28 reúne el registro de transferencia con el estado COMPENSADA y la publicación del evento y el registro de notificacion con el evento consumido junto con la bandeja de la cuenta de origen donde aparece el aviso de reversión.

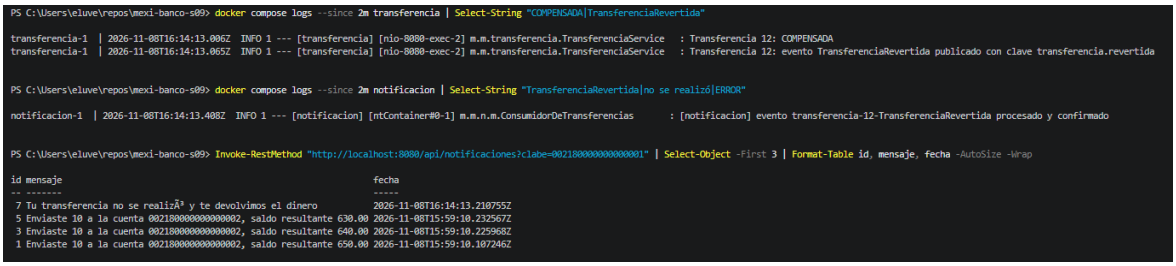


Figura 28. *Registros de transferencia y de notificacion con la transferencia 12 en COMPENSADA y la bandeja de la cuenta de origen con el aviso de reversión.*

Punto 3 · Usar ack manual y hacer el ack después de guardar

Activé `acknowledge-mode: manual` en `application.yml` y en el consumidor llamo a `basicAck` solo después de que `NotificacionService` guardó los avisos. El orden se ve en el código porque el guardado va antes del ack y también en el registro donde los avisos aparecen antes del mensaje "procesado y confirmado". La Figura 29 muestra nuevamente la configuración en modo manual.

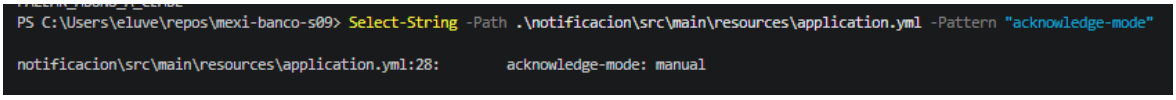


Figura 29. *Verificación de acknowledge-mode: manual en el application.yml de notificacion.*

RabbitMQ muestra el consumidor con `ack_required: True` y al terminar no quedan mensajes sin confirmar como se ve en la Figura 30 donde la cola tiene 0 mensajes listos y 0 sin confirmar con un consumidor activo. Si guardar falla la excepción sale antes del ack y el mensaje no se da por entregado y por eso no se pierde. Con ack automático el mensaje se confirmaría apenas llegara al consumidor y un fallo posterior haría perder el aviso.

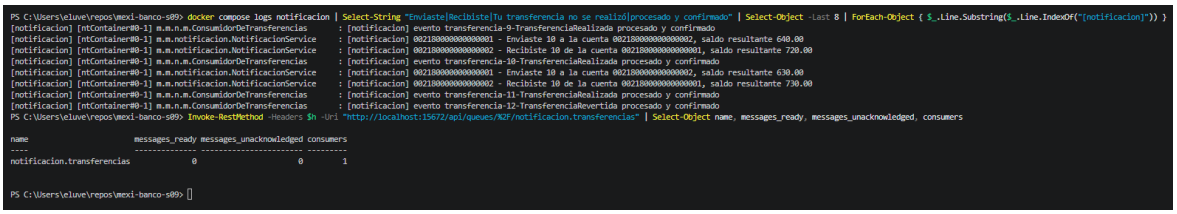


Figura 30. *Registro de notificacion con los eventos 9 a 12 procesados y confirmados y cola con 0 mensajes listos, 0 sin confirmar y 1 consumidor.*

El escenario completo que pide esta parte quedó cubierto con las capturas anteriores: tres transferencias con notificacion apagado (Figura 24) tres mensajes Ready en el panel (Figura 25) y la cola en 0 con los avisos en la bandeja al encender notificacion (Figura 26, Figura 27 y Figura 30).

Parte 3 · Rompe algo: el duplicado

La mensajería entrega “al menos una vez” y por eso un mismo evento puede llegar más de una vez. Esto ocurre por ejemplo cuando el consumidor se cae entre guardar y confirmar. En esta parte primero mostré el problema con avisos repetidos en la bandeja y después hice el consumidor idempotente.

Punto 1 · Provocar el duplicado

Publiqué dos veces en el exchange transferencias el mismo mensaje con la clave `transferencia.realizada` y el mismo `eventoId` (`transferencia-900-TransferenciaRealizada`) usando la API HTTP de RabbitMQ. Las dos publicaciones fueron enrutadas a la cola y notificacion procesó los dos mensajes y confirmó los dos de modo que la bandeja de la cuenta de origen quedó con el aviso “Enviaste 10… saldo resultante 620” repetido (ids 8 y 10). Esto pasa porque la mensajería entrega al menos una vez y el consumidor no distingue un evento que ya procesó.

En la Figura 31 se ven las dos publicaciones con `routed=True` y la bandeja con el aviso repetido y en la Figura 32 el registro con el mismo evento procesado y confirmado dos veces.

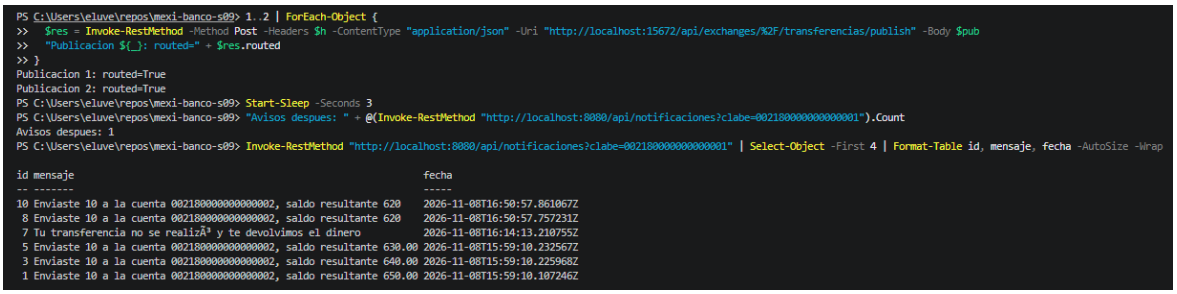


Figura 31. *Publicación doble del mismo mensaje y bandeja de la cuenta de origen con el aviso repetido (ids 10 y 8).*

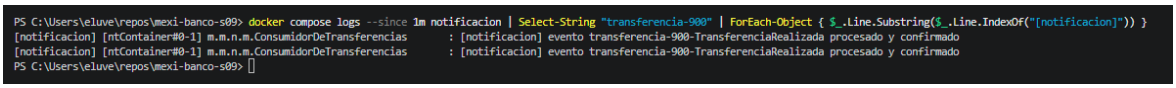


Figura 32. *Registro de notificacion con el evento transferencia-900-TransferenciaRealizada procesado y confirmado dos veces.*

Punto 2 · Hacer el consumidor idempotente

Para solucionarlo guardo el `eventoId` en una tabla con una restricción UNIQUE y lo hago en la misma transacción que los avisos. Si el evento ya estaba registrado no hago nada y confirmo el mensaje. Con ello el consumidor es idempotente porque el efecto de procesar un evento es el mismo sin importar cuántas veces llegue.

La Figura 33 muestra la estructura de la tabla `evento_procesado` con las columnas `id`, `evento_id` y `fecha`. En la sección Indexes aparece la línea `uk_evento_procesado_evento_id UNIQUE CONSTRAINT, btree (evento_id)` que demuestra que la restricción UNIQUE existe.

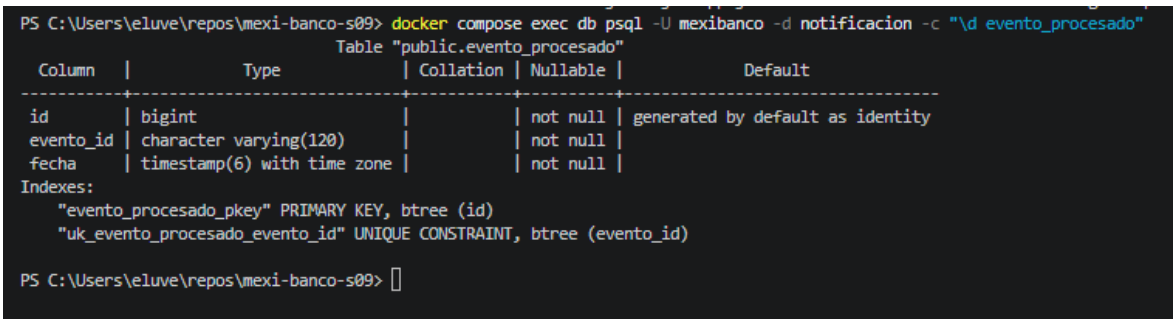


Figura 33. *Estructura de la tabla evento_procesado con su restricción UNIQUE sobre evento_id.*

Punto 3 · Repetir el experimento

Para repetir el experimento registré primero los conteos de avisos de cada cuenta (Figura 34) con 6 avisos en la cuenta de origen y 5 en la de destino. Después publiqué dos veces el mismo mensaje con el mismo `eventoId` (`transferencia-901-TransferenciaRealizada`).

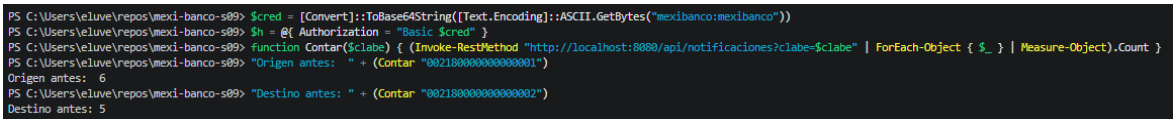


Figura 34. *Conteo de avisos de cada cuenta antes del experimento (origen 6 y destino 5).*

Esta vez notificacion procesó la primera copia y guardó el `eventoId` y los avisos en la misma transacción antes de confirmar el mensaje. Al llegar la segunda copia encontró el `eventoId` ya registrado y no guardó nada aunque confirmó igualmente el mensaje. Por eso la bandeja quedó con un solo juego de avisos (uno en la cuenta de origen y uno en la de destino) y la cola terminó en cero. Los conteos posteriores de la Figura 35 muestran 7 avisos en la cuenta de origen y 6 en la de destino y la Figura 36 muestra el procesamiento de la primera copia el mensaje que indica que la segunda ya estaba procesada y la tabla `evento_procesado` con una sola fila.

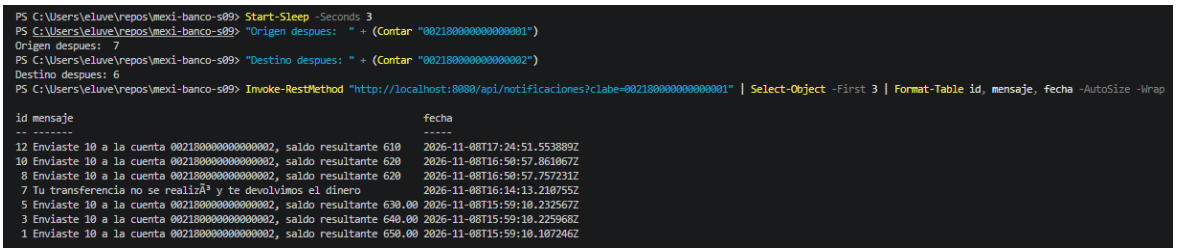


Figura 35. *Conteo de avisos después del experimento (origen 7 y destino 6) y últimos avisos de la cuenta de origen.*

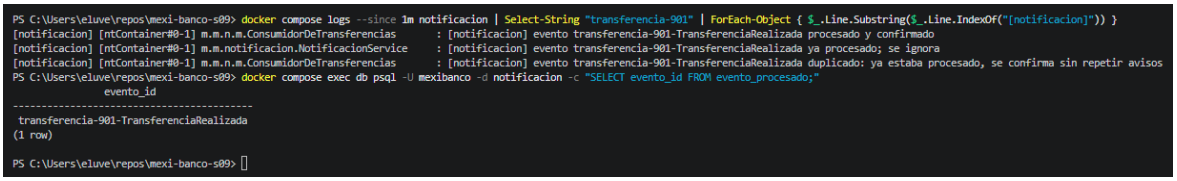


Figura 36. *Registro con el evento transferencia-901 procesado una vez y la segunda copia ignorada, y tabla evento_procesado con una sola fila.*

La restricción UNIQUE sobre `evento_id` funciona como última defensa porque si dos copias llegaran al mismo tiempo la segunda inserción fallaría y también se trataría como duplicado.

Parte 4 · Rompe algo: el mensaje envenenado

En esta parte publiqué en el exchange transferencias con la clave `transferencia.realizada` un cuerpo que no es un evento válido (`{"hola": }`) para observar qué ocurre antes y después de configurar la DLQ.

Punto 1 · Publicar un mensaje inválido y ver qué pasa antes de arreglarlo

Antes de configurar nada Spring rechazó el mensaje como error fatal y al no haber dead letter exchange el mensaje se descartó y se perdió. La cola quedó en cero y el consumidor siguió vivo y la única huella fue una línea en el registro.

La Figura 37 muestra la transferencia 15 que respondió 201 con estado COMPLETADA y cuyo evento `transferencia-15-TransferenciaRealizada` fue procesado y confirmado por notificacion.

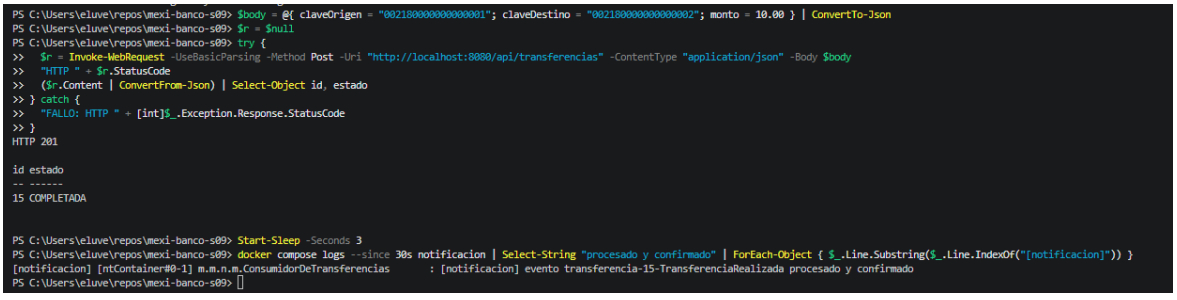


Figura 37. *Transferencia 15 con respuesta HTTP 201 y estado COMPLETADA y registro de su evento procesado y confirmado.*

Punto 2 · Configurar el DLX y la DLQ y el nack sin reencolar

Configuré en `RabbitConfig` el dead letter exchange `transferencias.dlx` la cola `notificacion.transferencias.dlq` y su binding y declaré la cola principal con los argumentos `x-dead-letter-exchange` y `x-dead-letter-routing-key`. En el consumidor leo el cuerpo del mensaje por mi cuenta y si no se puede convertir a evento o le faltan campos hago `basicNack` con `requeue=false` de modo que RabbitMQ lo reenvía a la DLQ. Así un mensaje envenenado deja de perderse en silencio y queda guardado para revisarlo.

Punto 3 · Borrar la cola vieja, reconstruir notificacion y repetir el experimento

RabbitMQ no permite cambiar los argumentos de una cola que ya existe y responde con `PRECONDITION_FAILED`. Por eso apagué notificacion y borré `notificacion.transferencias` por la API para después levantar el servicio con el jar nuevo que la declara de nuevo con `x-dead-letter-exchange` y `x-dead-letter-routing-key`.

La Figura 38 comprueba que la cola principal tiene esos argumentos y que existe la DLQ `notificacion.transferencias.dlq` y que el DLX está enlazado a ella.



Figura 38. *Argumentos de la cola principal, DLQ durable y vacía y binding de transferencias.dlx hacia la DLQ.*

Punto 4 · Comprobar que el mensaje malo está en la DLQ

Al repetir el experimento publiqué en el exchange transferencias el cuerpo inválido y el mensaje quedó en la DLQ con `reason: rejected` mientras que las transferencias normales siguieron generando avisos.

La Figura 39 muestra el mensaje obtenido de la DLQ con Get messages desde la API de RabbitMQ con su cuerpo la routing key `notificacion.transferencias.dlq` y el encabezado `x-death` que indica la cola de origen el motivo `rejected` y el exchange transferencias.



Figura 39. *Mensaje inválido en notificacion.transferencias.dlq con su encabezado x-death (reason: rejected).*

Para comprobar que el consumidor seguía funcionando realicé la transferencia 18 que respondió 201 y quedó en COMPLETADA (Figura 40).

```
PS C:\Users\eluve\repos\mexi-banco-s09> $body = @{ claveOrigen = "00218000000000001"; claveDestino = "00218000000000002"; monto = 10.00 } | ConvertTo-Json
PS C:\Users\eluve\repos\mexi-banco-s09> $r = $null
PS C:\Users\eluve\repos\mexi-banco-s09> foreach ($intento in 1..3) {
>>   try {
>>     $r = Invoke-WebRequest -UseBasicParsing -Method Post -Uri "http://localhost:8080/api/transferencias" -ContentType "application/json" -Body $body
>>     "Intento ${intento}: HTTP " + $r.StatusCode
>>     ($r.Content | ConvertFrom-Json) | Select-Object id, estado
>>     break
>>   } catch {
>>     "Intento ${intento}: FALLO HTTP " + [int]$_.Exception.Response.StatusCode
>>     Start-Sleep -Seconds 10
>>   }
>> }
Intento 1: HTTP 201

id estado
-- ------
18 COMPLETADA
```

Figura 40. *Transferencia 18 con respuesta HTTP 201 y estado COMPLETADA tras el mensaje inválido.*

La Figura 41 reúne el estado de la cola principal con 0 mensajes y 1 consumidor y el registro de notificacion donde aparece primero la advertencia de mensaje ilegible y después el evento de la transferencia 18 procesado y confirmado. Se incluye además la bandeja de avisos con el aviso de esa transferencia.

```
PS C:\Users\eluve\repos\mexi-banco-s09> Invoke-RestMethod -Headers $h -Uri "http://localhost:15672/api/queues/%2F/notificacion.transferencias" | Select-Object name, messages_ready, messages_unacknowledged, consumers

name                        messages_ready messages_unacknowledged consumers
----                        -------------- ---------------------- ---------
notificacion.transferencias             0                      0         1


PS C:\Users\eluve\repos\mexi-banco-s09> docker compose logs --since 10m notificacion | Select-String "transferencia-18|ERROR|Caused by|ilegible|procesado|duplicado" | ForEach-Object { $_.Line.Substring(0, [Math]::Min(240, $_.Line.Length)) }
notificacion-1  | 2026-11-08T22:59:19.496Z  WARN 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [notificacion] mensaje ilegible (Unexpected end-of-input within/between Object entries
notificacion-1  | 2026-11-08T22:59:57.816Z  INFO 1 --- [notificacion] [ntContainer#0-1] m.m.n.m.ConsumidorDeTransferencias       : [notificacion] evento transferencia-18-TransferenciaRealizada procesado y confirmado
PS C:\Users\eluve\repos\mexi-banco-s09> Invoke-RestMethod "http://localhost:8080/api/notificaciones?clabe=00218000000000001" | Select-Object -First 2 | Format-Table id, mensaje, fecha -AutoSize -Wrap

id mensaje                                                
```

Punto 1 · Provocar el problema

Apagué RabbitMQ con `docker compose stop rabbitmq` (Figura 42) y realicé una transferencia. La base de transferencia la dejó en COMPLETADA (id 19) porque el estado final se guarda antes de publicar el evento pero la publicación falló con `UnknownHostException: rabbitmq` y el cliente recibió un HTTP 500 tras 15 segundos de espera aunque su dinero ya se había movido (Figura 43).

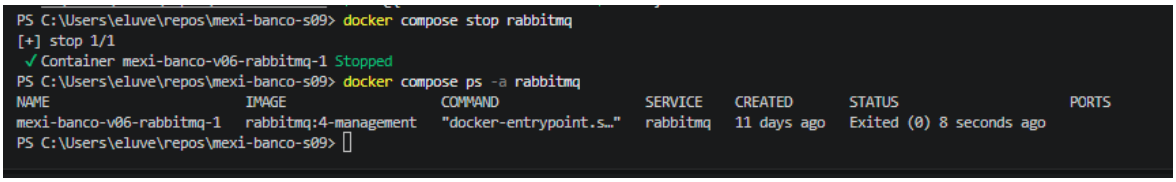


Figura 42. *Apagado del contenedor rabbitmq con docker compose stop y comprobación de su estado Exited.*

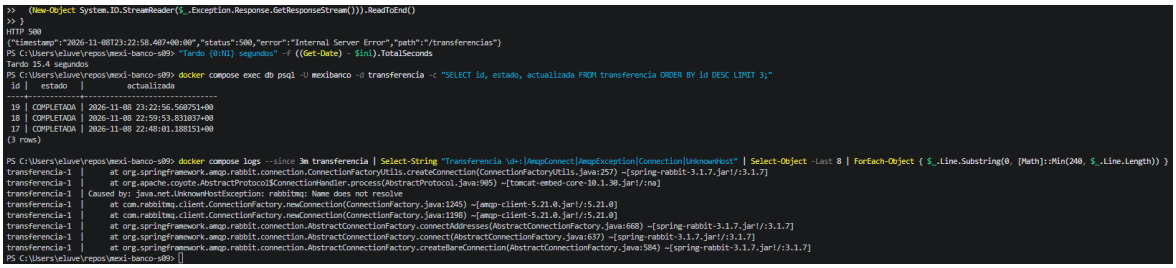


Figura 43. *Respuesta HTTP 500 tras 15.4 segundos, transferencia 19 en COMPLETADA en la base y UnknownHostException: rabbitmq en el registro.*

Además el evento `TransferenciaRealizada` nunca salió y por eso el cliente no recibió aviso aunque RabbitMQ volvió a encenderse (Figura 44). Es la doble escritura porque se escribe en la base y en el broker sin una transacción que abarque a los dos y si falla el segundo ambos quedan en desacuerdo.

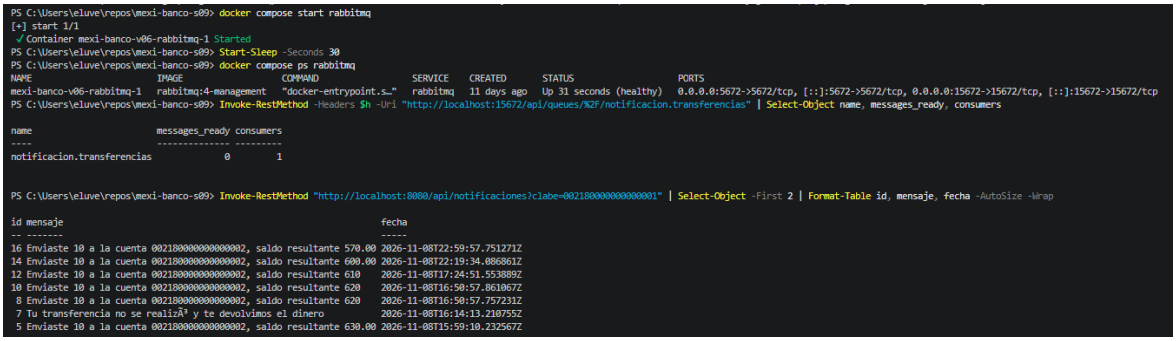


Figura 44. *Encendido de rabbitmq con la cola en 0 mensajes y 1 consumidor y bandeja sin ningún aviso nuevo para la transferencia 19.*

Punto 2 · Leer el código de transferencia

Este punto fue de solo lectura. Únicamente revisé el código del servicio de transferencia que se iba a modificar en los pasos siguientes y no realicé cambios.

Punto 3 · Crear la tabla outbox (entidad y repositorio)

Creé la tabla `outbox` junto con su entidad y su repositorio. La estructura de la tabla se muestra en la Figura 45 con las columnas `id`, `creada`, `evento_id`, `payload`, `publicada` y `routing_key`.

Punto 4 · Guardar el estado final y la fila del outbox en la misma transacción

Modifiqué el cierre de la saga para que el estado COMPLETADA y la fila del outbox con el evento se guarden en la misma transacción local. De esta manera el evento queda registrado para su publicación en cuanto la base confirma el cambio de estado.

Punto 5 · Escribir el relay @Scheduled con FOR UPDATE SKIP LOCKED

Escribí el relay con `@Scheduled` que publica después las filas pendientes y las marca como publicadas. La consulta utiliza `FOR UPDATE SKIP LOCKED` para el caso de que corra más de una réplica de transferencia y corre bien con una sola réplica.

Punto 6 · Ajustar los tests, generar el jar y reconstruir

Ajusté las pruebas y generé el jar para reconstruir después el contenedor de transferencia. La Figura 45 muestra la tabla `outbox` creada y el arranque de `TransferenciaApplication` en 18.676 segundos sin errores en el registro.

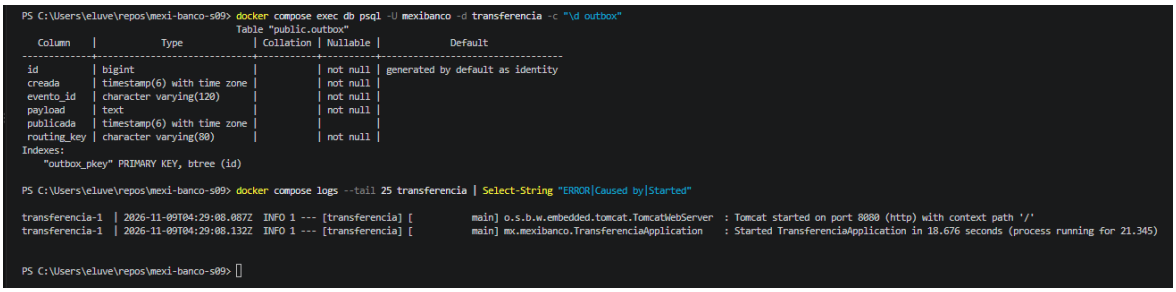


Figura 45. *Estructura de la tabla outbox y arranque de TransferenciaApplication sin errores tras reconstruir.*

Punto 7 · Probar con RabbitMQ apagado

Probé el escenario de la doble escritura apagando RabbitMQ y ejecutando un script que intenta la transferencia hasta cuatro veces con una pausa de 10 segundos entre intentos (Figura 46). Con el Transactional Outbox el estado COMPLETADA y el evento se guardan en la misma transacción local y un relay con `@Scheduled` publica después las filas pendientes.



Figura 46. *Apagado de rabbitmq y script que intenta la transferencia hasta cuatro veces con pausa de 10 segundos.*

Con RabbitMQ apagado la transferencia respondió 201 en el primer intento y quedó en COMPLETADA (id 22) como se ve en la Figura 47. En la Figura 48 se observa que tardó 3.5 segundos y que el evento quedó en la tabla `outbox` con `publicada = f` mientras que el registro de `OutboxService` confirma que la transferencia 22 se completó y que su evento se guardó en el outbox.

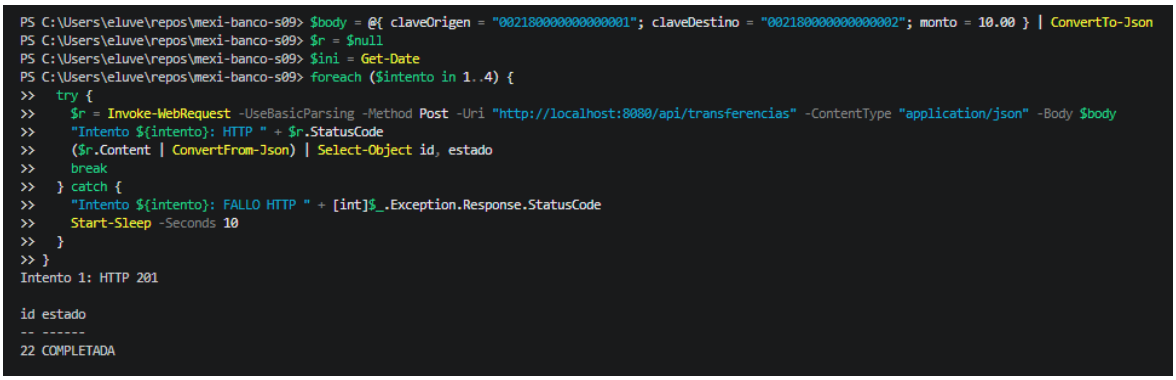


Figura 47. *Resultado del script con HTTP 201 en el primer intento y la transferencia 22 en estado COMPLETADA.*

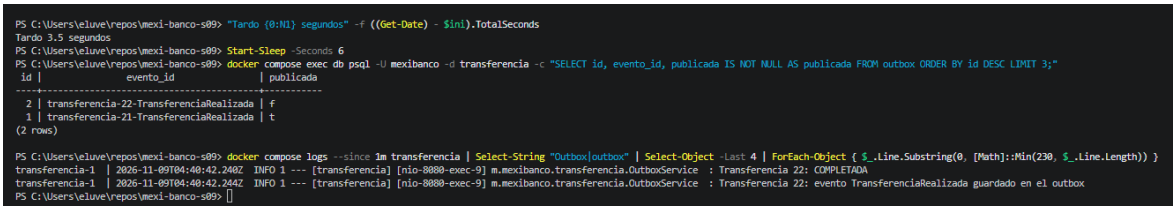


Figura 48. *Tiempo de 3.5 segundos, tabla outbox con la fila de la transferencia 22 con publicada = f y registro de OutboxService.*

Al encender RabbitMQ el relay publicó el evento sin intervención y notificacion lo procesó de modo que el aviso apareció en la bandeja. La publicación es “al menos una vez” porque el relay puede repetir un evento si se cae entre publicar y marcar la fila y eso es seguro porque notificacion es idempotente por `eventoId`. Antes del outbox en cambio la transferencia quedaba en COMPLETADA pero el cliente recibía un HTTP 500 tras 15 segundos y el evento nunca salía aunque RabbitMQ volviera a encenderse.

Límites de la solución

- **El outbox solo protege desde que existe.** La transferencia 19 que se completó con RabbitMQ apagado antes del outbox nunca tuvo aviso.
- **No cubre sagas cortadas a la mitad.** La transferencia 20 quedó en ABONADA por el 503 de antes y no tiene fila de outbox.
- **No probé SKIP LOCKED con dos réplicas.** La consulta con `FOR UPDATE SKIP LOCKED` corre bien pero con una sola réplica no se puede demostrar que dos instancias no publiquen la misma fila. El enunciado lo pide solo si se corre más de una réplica y por eso queda como implementado y no probado.

Preguntas de cierre

1. RabbitMQ caído cuando la saga llega a COMPLETADA

En la versión de la Parte 1 (sin outbox) la transferencia queda en COMPLETADA en la base porque el estado se guarda antes de publicar pero la publicación falla y el evento nunca sale. Lo comprobé al apagar RabbitMQ: la transferencia 19 quedó en COMPLETADA y el cliente recibió un HTTP 500 tras 15.4 segundos (`UnknownHostException: rabbitmq`) y aunque encendí RabbitMQ después ese aviso nunca llegó.

El publisher confirm no lo arregla porque solo responde si el broker recibió un mensaje que ya se le envió. Si el broker está caído no hay conexión que confirme nada y aunque la hubiera la confirmación no hace que el estado de la base y la publicación se confirmen juntos. El problema es la doble escritura que consiste en dos sistemas sin una transacción que los abarque y el confirm solo informa sobre uno de ellos.

En el reto lo resolví con un Transactional Outbox en el que COMPLETADA y el evento se guardan en la misma transacción local y un relay publica después. Con RabbitMQ apagado la transferencia 22 respondió 201 en 3.5 segundos y el evento esperó con `publicada = f` hasta que salió solo al encender RabbitMQ.

2. Por qué el ack va después de guardar

Si hago el ack antes de guardar RabbitMQ da el mensaje por entregado y lo borra. Si notificacion se cae justo después del ack y antes de guardar el aviso se pierde para siempre. Esa es una garantía “a lo sumo una vez” en la que el mensaje nunca se duplica pero puede perderse.

Si hago el ack después de guardar un fallo en medio hace que RabbitMQ reentregue el mensaje de modo que nunca se pierde pero puede procesarse dos veces. Esa es la garantía “al menos una vez” que fue la que elegí porque perder un aviso de dinero me parece peor que recibir uno repetido.

El costo apareció en la Parte 3 cuando publiqué dos veces el mismo `eventoId` y la bandeja quedó con avisos repetidos (ids 8 y 10). Para eso hice el consumidor idempotente guardando el `eventoId` con una restricción UNIQUE en la misma transacción que los avisos. Aclaro que el duplicado lo provoqué publicando dos veces desde la API de RabbitMQ y no matando al consumidor entre guardar y confirmar y por eso ese caso exacto no lo reproduje.

3. Elección y sacrificio

Elección: reemplacé la llamada HTTP síncrona a notificacion por eventos en RabbitMQ publicados con un outbox.

Porque: de esta forma la transferencia ya no depende de que notificacion o RabbitMQ estén vivos en ese momento y el aviso deja de poder tumbar una transferencia cuyo dinero ya se movió como pasó con las transferencias 6 y 7.

Sacrifico: la inmediatez y la certeza. El 201 ya no garantiza que el aviso se mandó y el aviso llega después (las transferencias 9 a 11 solo generaron avisos al encender notificacion). Además puede llegar repetido y exige idempotencia, DLQ y outbox lo que significa más piezas que operar. A esto se suma que el outbox no rescata sagas cortadas a la mitad como la transferencia 20 que quedó en ABONADA.